

TrueScout. WC26

Math · Engine · Design
under one lens.

A forensic audit of the **TrueScout-WC26** soccer analytics dashboard — the Bayesian rating engine, the Monte Carlo tournament simulator, the nightly ETL pipeline, the Next.js frontend, and the live UX at **true-scout.vercel.app** — evaluated against the product's stated audience: **World Cup 2026 casuals, enthusiasts, and new fans**. Findings are cited to file:line; fixes are prioritized by impact and effort; a 30 / 60 / 90-day roadmap leads to the knockout stage.

6.0

MATH

6.5

ENGINE

4.0

DESIGN

3.2

FAN-FIT

4.9

COMPOSITE

PREPARED BY

Z.ai Senior Analytics Consultancy

METHOD

30+ Python files · 40 Next.js files · 38 live screenshots

Table of Contents

Executive Summary

Master Scored Rubric

Audit Methodology

What was reviewed

Audience lens

Limitations

Part I — Math Audit

3.1 The Bayesian core is correct

3.2 [Major] The WC-floor override silently breaks the posterior

3.3 [Major] Brier tracker silently skips 90-minute draws

3.4 [Major] Monte Carlo is deterministic given point estimates

3.5 [Minor] Uncalibrated magic numbers throughout the math

3.6 FIFA-style 0-99 rating — the dual-display formula

Part II — Engine Audit

4.1 [Major] Team aliases duplicated in 5 places with diverging values

4.2 [Major] DuckDB write lock — nightly batch + FastAPI cannot coexist

4.3 [Major] run_date uses local date, not UTC

4.4 [Major] SQL-injection-shaped pattern in load_identity

4.5 [Major] No alerting, no checkpointing, empty stub directories

4.6 [Minor] Dead code & dependency hygiene

Part III — Design Audit

5.1 [Critical] Mobile sidebar does not collapse — three lines break the product

5.2 [Note] Knockout-only scope — intentional design decision

5.3 [Major] Jargon leaks everywhere despite a working Tooltip system

5.4 [Critical a l l y] LabelWithInfo icon is not keyboard-focusable

Part IV — Fan-Fit Audit

6.1 The 30-second onboarding test: failed

6.2 Position-label bug — visible credibility hit

6.3 Jargon inventory — the worst offenders

6.4 World Cup 2026 context fit — lean into the knockout narrative

Part V — Competitive Benchmark

Differentiation story for fans 16

Top 3 Wins17

W1. Backend: data-driven bracket + vectorised Monte Carlo + anti-hallucination LLM 17

W2. Frontend: loading skeletons on every route + bracket geometry 17

W3. Live site: About page is a masterclass + Compare delivers plain-English verdicts 17

Top 5 Fixes — After Overlap Check18

9.1 Already in-flight (from the PR plan — checked off) 18

9.2 Remaining gaps — the three items NOT in the PR plan 19

9.3 How to do each remaining fix 19

Tournament-Window Roadmap20

10.1 This week — the three remaining gaps (~11 hours) 20

10.2 Tournament-safe — pull back from deferred (ship during WC, ~12h) 21

10.3 Stay deferred — math core changes (ship after the tournament) 21

10.4 What NOT to build (ever) 22

Appendix22

A. Weighted Composite Score 22

B. Code Snippets Compendium 23

C. Closing Note 24

SECTION 1 · VERDICT

Executive Summary

TrueScout-WC26 is a technically ambitious soccer analytics dashboard whose component-level craft is genuinely senior-grade, but whose product-level execution currently fails the audience it was built for. The Bayesian rating math is clean at the algorithmic core but shipwrecks on a silent post-hoc override; the engine runs nightly without alerting; the Next.js frontend has every loading skeleton in place but cannot serve a phone; the live site hides its best feature behind an unguessable URL. Composite score: **4.9 / 10** — a strong demo that has not yet been hardened for casual fans.

The backend deserves real credit. The Normal-Normal conjugate Bayesian update (*bayesian_ratings.py:310-323*) is mathematically correct. The 10,000-iteration Monte Carlo (*monte_carlo_sim.py:521-535*) is vectorised in pure NumPy and runs in ~0.04 s. The bracket is reconstructed data-driven from ESPN fixtures, not hardcoded. An honest Brier-vs-market calibration loop exists. The LLM scouting reports use an anti-hallucination pattern with templated fact bullets and a jargon-ban constraint — Python does the math, the LLM does the prose. These are the right architectural choices.

But three live-site deal-breakers ship today. The sidebar does not collapse on mobile — a 224px-wide fixture eats 60% of a 375px iPhone, leaving the bracket to overflow by 973px and the dashboard cards to cramp into 55px of usable width. The AI Scouting Report on every player page returns HTTP 500 after ~45s of "Generating...", eroding trust faster than if the feature did not exist. The Compare tool — the single best feature on the site, delivering plain-English verdicts like "Messi has a marginal edge (0.24 pts)" — is invisible: not in the navigation, only reachable by typing /compare directly. Add a position-label bug that lists Vinícius Júnior as a "Full Back" on the homepage and a Brier page that promises "vs the bookies" with zero bookmaker data ingested, and the site is currently a polished demo that has not been hardened for the audience it claims to serve.

Master Scored Rubric

Scores are 0-10 with a one-line justification. The composite is a simple mean weighted equally across the three audit layers (backend, frontend, live site). "Fan-fit" is weighted double because it is the product's stated primary lens.

Dimension	Score	One-line justification
Math correctness	6.0	Conjugate Bayesian update is correct; WC-floor override (M1), Brier draw-skip (M2), prior-variance mis-spec (M3) bias published numbers.
Math rigor	4.0	HDI computed per-player but never propagated into team strength → title probs are point estimates in Bayesian clothing.
Engine architecture	7.0	Clean Bronze→Silver→Gold medallion; idempotent loaders; data-driven bracket. Loses points for singleton DuckDB write lock (E3) and 1,388-line monolithic export_json.py.
Pipeline robustness	7.0	Multi-source ingestion with TLS spoofing, retries, schema validation. Loses points for zero alerting and zero checkpointing (E7).

Dimension	Score	One-line justification
API design	5.0	Pydantic models + CORS, but no caching headers, no rate limits, sync blocking handlers, SQL-injection-shaped pattern in load_identity (E6). API not even deployed in prod.
Backend audience-fit	7.0	FIFA-style 0-99 scores, jargon-banned LLM prompts. Loses points for still exposing posterior_std / hdi_low / hdi_high to the API.
Visual design system	5.0	Coherent dark "scouting room" aesthetic; but no documented tokens, no WC 2026 brand identity, no type scale, sub-WCAG-AA muted text.
Onboarding for new fans	3.0	Homepage H1 = "Dashboard" with no value prop; About page is excellent but buried; xG/xA/Per 90/Chaos/Monte Carlo dropped without glossary.
Mobile UX	2.0	Fixed w-56 sidebar never collapses; bracket forces 1280×908px canvas; touch targets below 44px throughout. Mobile is essentially broken.
Performance / DX	6.0	Real skeletons everywhere, parallel Promise.all. Loses points for recharts + framer-motion overuse and 2-4 MB players.json fetched client-side on every search.
Accessibility	4.0	Semantic nav/main used; but LabelWithInfo icon not keyboard-focusable, th onClick sort headers have no aria-sort, no skip-to-content, no error.tsx.
WC 2026 context fit	3.0	Group stage ENTIRELY MISSING; no host-nation flavor despite backend having venues_2026.json; no matchday navigation.
Live site — first impression	5.0	Visually clean but H1="Dashboard"; model divergence (no Argentina in top-5, Cape Verde 45% vs Argentina) looks implausible without context.
Live site — mobile	2.0	Sidebar doesn't collapse; bracket overflows 973px on 375px viewport; multiple pages have horizontal scroll.
Live site — onboarding	3.0	No hero copy, no inline glossary, About orphaned from data cards. A casual fan cannot understand the product in 30s.
Live site — visual polish	6.0	Clean dark-mode design system, consistent emerald accent, dense but organized player detail. Loses points for duplicated-position rendering bug.
Live site — information architecture	5.0	Compare hidden from nav, Groups missing entirely, Brier/Calibration naming inconsistent, no matchup detail pages.
Live site — trust & transparency	7.0	About page is masterclass. Loses points for implausible Value Pick, 0 bookmaker data, and broken AI scouting report.

Composite (equal-weighted): 4.9 / 10 · Fan-fit weighted 2x: 4.3 / 10 · Closing the three remaining gaps (mobile sidebar, Compare-in-nav, Brier draw-skip) plus fixing the position-label bug would lift this to ~6.5.

Three Critical Findings (must ship before casual-fan launch)

1. Mobile sidebar does not collapse — 224px on 375px viewport; bracket overflows by 973px. 2. AI Scouting Report returns HTTP 500 on every player page after ~45s of "Generating...". 3. Compare is hidden from navigation — only reachable by typing /compare. Fix in the tournament-window sprint (\$9-10).

SECTION 2 · SCOPE & METHOD

Audit Methodology

This audit was conducted over a single session on 2026-07-01 against the public GitHub repository `github.com/shehzanwar/TrueScout-WC26` and the live deployment at `true-scout.vercel.app` (data run label: 2026-06-30). Three parallel investigations were run by specialised subagents and synthesised here.

What was reviewed

Audit track	Method	Files / surfaces covered
Backend math & engine	30+ Python files read end-to-end	<code>bayesian_ratings.py</code> , <code>monte_carlo_sim.py</code> , <code>brier_tracker.py</code> , <code>feature_selection.py</code> , <code>archetypes.py</code> , <code>generate_narratives.py</code> , <code>build_features.py</code> , all loaders, all source pulls, all API routes, <code>run_nightly.py</code> , <code>requirements.txt</code> , <code>ARCHITECTURE.md</code> , <code>PRD.md</code> , <code>CONTEXT.md</code>
Frontend design & UX	40 Next.js files read end-to-end	All <code>app/*</code> routes, all <code>components/</code> , all <code>lib/*</code> utilities, <code>layout.tsx</code> , <code>globals.css</code> , <code>package.json</code> , <code>next.config.ts</code> , <code>README</code> , <code>CLAUDE.md</code> , <code>AGENTS.md</code>
Live site walkthrough	38 screenshots, 2 viewports	Headless Chrome on desktop 1280×800 and mobile 375×812 (iPhone 12). Every nav item, every player detail variant, mobile overflow measurements per page, keyboard-only navigation pass.
Competitive benchmark	Mental model vs 4 incumbents	SofaScore (live tracker + breadth), FotMob (news + matchday), FBref (sortable tables + shot data), Understat (xG-only deep stats). Differentiation story evaluated for the WC26 casual-fan audience.

Audience lens

The product's stated primary audience is **World Cup 2026 casuals, enthusiasts, and new fans** — not stats PhDs. Every finding in this audit is filtered through that lens. A mathematically correct feature that a casual fan cannot interpret is scored as a partial failure. A polished UX that breaks on mobile is scored as a critical failure because the majority of WC fans arrive from social on phones. The audit tone is intentionally rigorous — senior consultant mode — because the product owner asked for scrutiny, not encouragement.

Limitations

All backend findings are static-code findings; no runtime verification was performed against DuckDB or the live API (which is not deployed — the frontend reads static JSON exported by the nightly run). Where a finding depends on runtime behavior (e.g. "the WC floor fires for these specific players"), the audit marks it as "needs runtime check" or qualifies the claim. The mobile-overflow measurements are real — captured via headless Chrome `scrollWidth` inspection on a 375×812 viewport. The AI Scouting Report 500 error was verified live on Lionel Messi's player page and reproduced on retry.

SECTION 3 · BAYESIAN RATINGS, MONTE CARLO, BRIER

Part I — Math Audit

The math is genuinely well-architected at the algorithmic level — a clean Normal-Normal conjugate Bayesian update, a vectorised NumPy Monte Carlo, an honest Brier-vs-market calibration loop. The cracks are in (a) ad-hoc post-hoc overrides that break the Bayesian framework's internal consistency, (b) silent sample-selection bias in Brier scoring, (c) zero uncertainty propagation from per-player posteriors into title probabilities. The dashboard's headline numbers are point estimates wearing a Bayesian hat.

3.1 The Bayesian core is correct

The Normal-Normal conjugate update (*bayesian_ratings.py:310-323*) is the mathematical heart of the system, and it is implemented correctly. Precisions add; the posterior mean is the precision-weighted average of the prior mean and the World-Cup observation mean; the no-WC-data case is handled cleanly via `np.where(np.isinf(sigma2_wc), 0.0, ...)` which forces $\tau_{wc} = 0$ and lets the posterior equal the prior. This is textbook partial pooling.

```
# etl/models/bayesian_ratings.py:310-323
tau_prior = 1.0 / sigma2_prior
tau_wc     = np.where(np.isinf(sigma2_wc), 0.0, 1.0 / sigma2_wc)
tau_post   = tau_prior + tau_wc

mu_prior = df["prior_mean"].values
wc_obs_col = "wc_rating_adjusted" if "wc_rating_adjusted" in df.columns else "wc_rating_avg"
mu_wc = df[wc_obs_col].fillna(df["wc_rating_avg"]).fillna(df["prior_mean"]).values

# Posterior
mu_post = (tau_prior * mu_prior + tau_wc * mu_wc) / tau_post
sigma2_post = 1.0 / tau_post
```

Snippet 1 — The Bayesian update (correct conjugate math). This is the part that is right.

3.2 [Major] The WC-floor override silently breaks the posterior

The single most consequential line in the codebase is in *monte_carlo_sim.py:321-329*. Team strength is computed from `GREATEST(posterior_mean, wc_rating_avg * 0.90)` rather than from the posterior itself. The docstring justifies this as a guard against MLS / non-European players whose club priors are absent and would otherwise be dragged below their demonstrated WC level — but that is exactly the problem the Bayesian model was supposed to solve by setting `tau_wc` high when minutes are low. Bypassing the posterior with a `GREATEST` floor means: (i) the model's stated uncertainty is discarded, (ii) two players with identical posteriors but different WC raw ratings produce different team strengths, and (iii) the Monte Carlo title probabilities no longer reflect the Bayesian model — they reflect a hybrid that nobody has formally specified.

```
# etl/models/monte_carlo_sim.py:321-329
wc_floor_expr = f"""
GREATEST(
    pr.posterior_mean,
    CASE WHEN pr.wc_minutes >= 90 AND f.wc_rating_avg IS NOT NULL
        THEN f.wc_rating_avg * 0.90
        ELSE pr.posterior_mean
    END
)"""
```

Snippet 2 — The WC-floor hack. Every title probability on the dashboard flows through this expression.

Why it matters: the headline "France 7.1% to win" numbers are produced by this hybrid, not by the Bayesian model the PRD advertises. A fan comparing TrueScout to market odds is comparing apples to a fruit salad.

Fix: either remove the floor and accept that low-prior players get shrunk toward the archetype mean (the Bayesian-correct behavior), or make the prior for no-club-data players wider so the WC likelihood dominates naturally — do not post-process the posterior. If the floor must stay, document it in the PRD as a known deviation and emit two columns (`posterior_mean` and `effective_strength`) so the substitution is auditable.

3.3 [Major] Brier tracker silently skips 90-minute draws

In a knockout match that ended 1-1 at 90 minutes, both the model and the market offered a 2-way "to-advance" probability. The actual outcome — who advanced via extra time or penalties — is observable, but the Brier tracker skips the match entirely because ESPN's `home_score` / `away_score` reflect only 90 minutes. This biases the Brier score upward because the skipped matches are exactly the closest ones — where the model and market were least confident — so the surviving sample over-represents blowouts where everyone gets the prediction right.

```
# etl/models/brier_tracker.py:250-257
else:
    # 90-min draw – in knockout this means ET/pens; scores reflect 90 min
    # We don't have the ET/pens result in Bronze yet – log and skip
    logger.warning(
        "%s vs %s ended %d-%d (90 min draw) – ET/pens result not in Bronze yet, skip
ping.",
        home, away, h_score, a_score,
    )
    continue
```

Snippet 3 — The Brier draw-skip. The data is already in Sofascore's EVENTS_SCHEMA — the tracker just doesn't use it.

Why it matters: the Brier score the dashboard advertises is not a calibration metric over all knockout matches — it is a calibration metric over *decisive 90-minute* matches. As the tournament progresses and more matches go to ET/pens, this bias will worsen. Fans comparing "TrueScout Brier 0.17" to "market Brier 0.19" are being misled. **Fix:** pull ET/pens outcomes from Sofascore's `home_score_et` / `away_score_et` / `home_score_penalties` / `away_score_penalties` columns — they are already in the EVENTS_SCHEMA at `sofascore_pull.py:184-189`. Encode outcome = 1 if home advances after ET/pens, 0 otherwise. The current outcome encoding is correct; the bug is the `continue` rather than resolving the post-90-min winner.

3.4 [Major] Monte Carlo is deterministic given point estimates

The Monte Carlo samples *match outcomes* given a fixed 32-vector of team strengths — it does not sample *team strength* from each player's posterior. The 10,000 iterations give the illusion of uncertainty quantification, but the uncertainty is only over match outcomes conditional on point-estimate team strengths. The 90% credible intervals on player posteriors are computed and stored in DuckDB but never enter the simulation.

```
# etl/models/monte_carlo_sim.py:521-535 — the vectorised Monte Carlo core
for round_idx in range(1, n_rounds):
    n_alive = current.shape[1]
    n_matches = n_alive // 2
    left = current[:, 0::2]    # (n_sim, n_matches)
    right = current[:, 1::2]
    s_left = strengths[left]   # NumPy fancy indexing → same shape
    s_right = strengths[right]
    p_left = 1.0 / (1.0 + np.power(10.0, -(s_left - s_right) / scale))
    rand = rng.random((n_sim, n_matches))
    winners = np.where(rand < p_left, left, right)
    current = winners
```

Snippet 4 — The vectorised Monte Carlo core. Elegant and fast — but `strengths` is a deterministic 32-vector, not a posterior sample.

Why it matters: the dashboard's title probabilities are sharper than the underlying data warrants. A team whose top-15 average is 7.83 ± 0.05 is treated identically to a team whose top-15 average is 7.83 ± 0.30 . The PRD's "honest uncertainty" promise is broken at the most visible layer. **Fix:** per sim iteration, sample each team's strength from `Normal(team_strength_mean, team_strength_std)` where `team_strength_std` is derived from the per-player posterior stds (e.g. `sqrt(sum(sigma_i^2 for top-15)) / 15`). Adds one line and roughly 0.01 s to the sim runtime. Sample sizes will become honestly wider for sparse-WC teams — which is exactly what the PRD promised.

3.5 [Minor] Uncalibrated magic numbers throughout the math

Five constants drive the math and none are calibrated. The BOARD.md backlog acknowledges empirical calibration is "blocked on ≥ 20 graded knockout matches" — but in the meantime, every published number is sensitive to these choices.

Constant	Location	Value	Why it matters
PRIOR_PULL	bayesian_ratings.py:289	0.50	Regression-to-the-mean dampening on club-prior → WC-prior translation. Could equally be 0.30 or 0.70.
BASE_WC_VAR	bayesian_ratings.py:308	90.0	Variance of a single 90-minute WC rating observation. Assumes ratings are i.i.d. per match — subs violate this.
LOGISTIC_SCALE	monte_carlo_sim.py:530	~0.5	Logistic scaling on strength differential → win probability. Uncalibrated vs historical upset rates.
TOP_N_PLAYERS	monte_carlo_sim.py:63	15	Team strength = mean of top-15 posteriors. Why 15 not 11? Squad-depth reward or oversight? Not documented.

Constant	Location	Value	Why it matters
ET_BIAS (3 versions)	brier_tracker.py:57 / matchups.py:209 / export_json.py:413	0.55/0.45 vs 0.5	The Brier page and the matchups page disagree on the same match's market probability.

Fix: expose each as a config setting + CLI flag for sensitivity analysis. When ≥20 graded matches exist, grid-search the parameters. Pick one ET-bias convention and import it everywhere.

3.6 FIFA-style 0-99 rating — the dual-display formula

The FIFA-style 0-99 score shown alongside the 1-10 Bayesian posterior is a blend of **absolute skill** (the posterior mean, normalized) and **relative rank** (position-group percentile). This mirrors the FIFA video-game scale: an 89-rated CB is both very good in absolute terms AND a top-decile centre back. The implementation lives in `etl/export_json.py:574-583` and is documented in the attached PR plan's PR 6.1.

```
# etl/export_json.py:574-583
def _fifa_score(posterior_mean, percentile_rank):
    # 60% absolute + 40% relative, clamped to [10, 99]
    if posterior_mean is None or percentile_rank is None:
        return None
    absolute = 10 + (posterior_mean - 1.0) * (89.0 / 9.0)
    relative = 10 + percentile_rank * 89.0
    return int(round(max(10.0, min(99.0, 0.60 * absolute + 0.40 * relative))))
```

The FIFA-style score: 60% absolute posterior + 40% position percentile, clamped to [10, 99].

Why 60/40: heavily weighting position percentile would give the best Bolivia midfielder a 99 (he is top of his position group nationally), which is wrong. Weighting absolute too high erases positional context (a 6.5-rated DEF is decent; a 6.5-rated FWD is poor). 60/40 balances them. The bands are: **90+ World Class, 85-89 Elite, 80-84 Top Tier, 75-79 Quality, 70-74 Good, 65-69 Decent, <65 Squad** — defined in `_FIFA_BANDS` at `export_json.py:563-571`. The same formula is applied per-radar-axis to produce sub-attribute scores (SHO / PAS / DEF / WC_FORM for outfielders; DIV / HAN / POS / KIC for goalkeepers) via `_fifa_attrs()` at line 595.

Audit note: the formula is clean and the 60/40 split is well-reasoned. The one gap is that `percentile_rank` is computed within a position-group that is itself contaminated by the position-misclassification bug (§6.2) — so a player bucketed as "DEF" when they are actually a "FWD" gets ranked against the wrong peer set, distorting both the percentile and the resulting FIFA score. The attached PR plan's PR 1.1 (position fallback fix with modal Sofascore position + UNK bucket + GK/xG sanity asserts) addresses this root cause. Once that ships, the FIFA scores will be internally consistent.

SECTION 4 · PIPELINE, API, DATA LINEAGE

Part II — Engine Audit

The engine is more mature than the math. Clean Bronze→Silver→Gold medallion, idempotent loaders, data-driven bracket reconstruction, multi-source ingestion with TLS spoofing and rate-limit retries. The losses are in single-source-of-truth violations (team aliases duplicated in 5 places), a singleton DuckDB write lock that prevents nightly batch + FastAPI coexistence, and zero alerting or

checkpointing on the nightly run.

4.1 [Major] Team aliases duplicated in 5 places with diverging values

team_aliases.py defines the canonical 14-entry `TEAM_ALIASES` dict. *monte_carlo_sim.py* and *export_json.py* import it correctly. But *brier_tracker.py*:65-75 and *matchups.py*:59-67 each define their own 7-entry `_NAME_ALIASES` dict that is a stale subset. The canonical map has 14 entries; the stale copies have 7. Missing from the stale copies: "Czechia" → "Czech Republic", "Holland" → "Netherlands", "Korea Republic" → "South Korea", "KSA" → "Saudi Arabia", "United Kingdom" → "England".

Why it matters: a Czech player's rating joins cleanly in the Bayesian step (uses canonical aliases) but the same player's Brier score and matchups page can fail to resolve the team because the stale copies do not include Czechia. The dashboard will silently show a mismatch between "model P(advance) = 0.42" (sim) and "market P(advance) = N/A" (matchups) for the same match. **Fix:** delete the duplicate dicts in *brier_tracker.py*:65-75 and *matchups.py*:59-67. Import `TEAM_ALIASES` and `normalize` from `etl.utils.team_aliases` instead. One-line fix per file.

4.2 [Major] DuckDB write lock — nightly batch + FastAPI cannot coexist

The write connection is a singleton guarded by a `threading.Lock`. `get_read_conn()` returns a *cursor* on the write connection — not a separate read-only connection. Any write transaction blocks all reads. The `CONTEXT.md` explicitly warns "Never run the nightly batch while the FastAPI server is live" — but this is a documentation-level mitigation, not a code-level one. A user opening the dashboard mid-batch gets either a blocked request (the sync FastAPI handler hangs until the lock releases) or a `duckdb.IOException: Conflicting lock`. For single-user V1 this is tolerable; for any multi-user scenario it is a non-starter. **Fix:** add a programmatic check at the top of *run_nightly.py* that fails fast if the FastAPI server is running. For V2, snapshot the `.duckdb` file after each nightly run and serve reads from the snapshot.

4.3 [Major] run_date uses local date, not UTC

monte_carlo_sim.py:760 uses `date.today()` — which returns the server's local date. The rest of the pipeline uses `datetime.now(timezone.utc)`. If the CI runner is in UTC and the dev machine is in PST, a sim run at 11pm PST on July 5 produces `run_date = 2026-07-05`, while the same sim run on CI at 7am UTC on July 6 produces `run_date = 2026-07-06`. The `simulations` table primary key is `(run_date, round, team_id)`, so these are treated as two different runs. The API filter `WHERE run_date = (SELECT MAX(run_date) FROM simulations)` returns whichever run is "latest" by date — which may be the dev run, not the CI run that produced the JSON the frontend is actually displaying. **Fix:** one line — replace `date.today()` with `datetime.now(timezone.utc).date()`.

4.4 [Major] SQL-injection-shaped pattern in load_identity

load_identity.py:159-165 f-string-interpolates `reep_id` and override values into SQL. The overrides file is committed to git (`data/static/position_overrides.json`), so this is not currently exploitable — but if a `reep_id` ever contains a single quote (theoretical for a hash-based ID, but unlikely) or the JSON is hand-edited with a typo, the SQL breaks or worse. This is a bad pattern even when not exploitable. A senior reviewer would flag this in any PR. **Fix:** use parameterised queries: `conn.execute("UPDATE`

```
identity_players SET position = ?, position_detail = ? WHERE reep_id =
?", [fields.get("position"), fields.get("position_detail"), reep_id]).
```

4.5 [Major] No alerting, no checkpointing, empty stub directories

run_nightly.py runs 11 steps sequentially, logs successes/failures to `logs/nightly.log`, and exits 1 on critical failures. There is no email/Slack alerting — the only way to notice a failure is to manually check the log or watch the GitHub Actions badge. There is no checkpointing — if step 7 (Monte Carlo) fails, step 8 (Brier) still runs against stale simulation data, and step 9 (export) re-exports the stale data to JSON. A silent failure in the Monte Carlo step (e.g. ESPN renames "Round of 32" to "Round of 32 (knockout)" and the bracket loader raises `RuntimeError`) propagates stale title odds to the live dashboard. Fans see yesterday's odds labelled as today's.

Compounding this: the `etl/matching/` directory contains only an empty `__init__.py`. The PRD and ARCHITECTURE.md reference "Phase 2 ID resolution" in `etl/matching/` (e.g. *load_club_priors.py:12-14* says "Run AFTER: `python -m etl.matching.resolve_understat`"), but no such module exists. The Understat→Reep resolution is done inline in *understat_pull.py:253-375* instead. The ghost directory signals an abandoned refactor. **Fix:** add a Slack/Discord webhook call in `run_nightly.main()` on critical failure; add a `--since-step` flag for resumable runs; either delete `etl/matching/` or move the Understat resolution logic there as advertised.

4.6 [Minor] Dead code & dependency hygiene

Three dependencies in `requirements.txt` are never imported: `pymc` (the Bayesian model is pure NumPy — the file header explicitly says "NO PyMC, NO NumPyro, NO MCMC"), `apscheduler` (the scheduler is GitHub Actions + Task Scheduler), and `rich`. `pymc` alone pulls in PyTensor/Aesara and adds ~200 MB to the install. Meanwhile *fbref_pull.py* is 916 lines of working code for a source that has been "permanently dead" since Opta/Stats Perform pulled data in Jan 2026 per the PRD — preserved "for reference only" but not gated against accidental invocation. All dependencies use `>=` pins with no upper bound, so a breaking change in any of them would silently break the pipeline on the next CI run. **Fix:** remove the three unused deps; move *fbref_pull.py* to `etl/sources/_archive/`; pin critical-path dependencies with `~=` or generate a `requirements.lock` via `pip-compile`.

SECTION 5 · FRONTEND CRAFT, MOBILE, A11Y

Part III — Design Audit

The frontend is genuinely well-engineered at the component level — clean server/client split, real loading skeletons on every route, thoughtful tooltip system, coherent dark "scouting room" aesthetic. But it fails its primary audience on three counts that matter more than the polish: (1) the WC 2026 group stage is missing, (2) the fixed sidebar does not collapse on mobile, (3) stat jargon is dropped onto fans with no inline glossary despite a perfectly-built Tooltip component being available. It reads like a generic SaaS analytics dashboard wearing a "WC 2026" sticker.

5.1 [Critical] Mobile sidebar does not collapse — three lines break the product

The single biggest UX defect in the product is three lines of code in *layout.tsx:31-33*. The body is a flexbox row at all viewports. The sidebar is a fixed **w-56** (224px) with no **hidden lg:flex**, no hamburger, no drawer. On a 375px iPhone the sidebar eats 60% of the viewport, leaving the main content squeezed into 151px (minus **p-6** = 96px padding) = 55px of actual content width.

```
// app/layout.tsx:31-33 — the broken mobile layout in three lines
```

```
{children}
```

Snippet 5 — The most consequential three lines in the codebase. `` never collapses; on 375px viewport the main content is squeezed to 55px.

The bracket page overflows by **973px** on a 375px viewport — the bracket visualization is desktop-only with no mobile fallback. The Messi player detail page overflows by 105px. The Brier match log table overflows by 78px. The matchups page overflows by 47px. Only the player search page fits cleanly. Given that 60%+ of WC fans arrive on mobile, this is the single highest-ROI fix on the site. **Fix:** add a `SidebarClient.tsx` with `useState(isOpen)`. Render `<aside className="hidden lg:flex">` for desktop and a `<button className="lg:hidden">` hamburger + drawer overlay for mobile. Use `usePathname` to auto-close on navigation.

5.2 [Note] Knockout-only scope — intentional design decision

The dashboard covers the knockout rounds only (R32 → R16 → QF → SF → F). This is an **intentional scoping decision** by the developer, not an oversight. The rationale is sound: the 48-team group stage involves many nations whose players and leagues lack the Understat / FBref / Sofascore coverage needed for the Bayesian prior, so rating them would produce low-confidence outputs that dilute the product's credibility. Focusing on the 32-team knockout bracket — where every team has at least 3 group-stage matches of WC data plus club priors for most players — keeps the rating engine in its high-coverage regime. This audit **accepts this scoping decision** and does not recommend adding a groups page. The one residual risk: a casual fan who arrives during the group stage (the first ~2 weeks) finds nothing to engage with. A lightweight mitigation — a single "Group stage in progress — knockout bracket activates June 27" banner on the homepage — costs ~30 minutes and sets expectations without requiring group-stage data infrastructure.

5.3 [Major] Jargon leaks everywhere despite a working Tooltip system

The `LabelWithInfo` abstraction (*Tooltip.tsx*) is the right pattern for jargon — a tooltip-wrapped label with an inline ⓘ icon. It is used in perhaps 4 places. Meanwhile *RawStats.tsx:147-178* shows bare string labels — "xG", "xA", "Shots", "Key passes", "Tackles", "Interceptions", "Clearances", "Saves" — none with tooltips. The "Per 90 / Totals" toggle, the single most confusing UX for casuals, has no explanation of what "per 90" even means. xG and xA are the two stats casual fans most often ask "what does that mean?" — and they are the least labeled.

On the bracket page, *BracketGrid.tsx:140* shows `title={`Chaos: avg match entropy = ${...}%`}`. "Chaos" (informal) and "entropy" (physics) both appear with no plain-English bridge. A casual fan reading "avg match entropy = 47%" has no idea whether 47 is high, low, or desirable. The AI Scouting Report subtitle reads "AI-generated scouting report · OpenRouter" — OpenRouter is the LLM router vendor; casual fans do not know or care. The sidebar entry "Model Calibration" links to `/brier` — naming inconsistency. The Brier page's "Δ Brier" column header is unexplained. The About page is the only place many of these terms are defined, and it is buried at the bottom of the sidebar in `text-xs text-slate-600`.

5.4 [Critical a11y] LabelWithInfo icon is not keyboard-focusable

The `LabelWithInfo` info icon is a `<span>` inside a `<span>` with `onMouseEnter/onMouseLeave/onFocus/onBlur/onClick` — but it is not a `<button>` and has no `tabIndex`. Keyboard users cannot trigger any of the tooltips the audit recommends adding. Compounding: *MatchLogTable.tsx:114-145* uses `<th onClick>` sort headers with no `aria-sort`, no keyboard handler. There is no skip-to-content link. Disclosure buttons (lineups, advanced metrics, alts) lack `aria-expanded`. Flag emoji have no `aria-label`. Color-only differentiation for ratings (green ≥ 7.0 / amber 6.0-7.0 / red < 6.0) fails for the 8% of men with red-green colorblindness. **Fix:** wrap the SVG in `<button type="button" aria-label={`More info: ${label}`}>`; convert sort headers to `<button>` with `aria-sort`; add a skip-to-content link; add secondary text signals (▲/●/▼) next to color-coded ratings.

SECTION 6 · ONBOARDING, JARGON, WC26 CONTEXT

Part IV — Fan-Fit Audit

Through the lens of the stated primary audience — WC 2026 casuals, enthusiasts, and new fans — the product currently fails its 30-second comprehension test. The homepage H1 is "Dashboard" with no value prop. The About page (genuinely excellent — Bayesian shrinkage in plain English, 5 honest limitations) is buried at the bottom of the sidebar. A position-label bug lists Vinícius Júnior as a "Full Back" on the homepage. The jargon inventory runs to 20+ unexplained terms. None of this is unfixable — the Tooltip system exists, the About page exists, the position data is correct in the detail page — but the wiring is missing.

6.1 The 30-second onboarding test: failed

When a brand-new fan lands on `true-scout.vercel.app`, the H1 is literally "Dashboard". The subhead is either "10,000 simulations · run 2026-06-30" or "Live model output from the nightly batch". Neither tells a brand-new visitor what TrueScout is, what they can do, or why they should care. There is no "Start here" CTA, no first-time tour, no value prop. A casual fan who clicks a Twitter link bounces within 10 seconds. **Fix:** replace "Dashboard" H1 with "Your World Cup 2026 prediction cockpit" (or similar), add a one-sentence explainer above the cards, and surface a "New here? Take the 30s tour →" pill linking to `/about`. Promote a "★ How TrueScout works" pill to the top of the sidebar (above Dashboard) for first-time visitors, gated via `localStorage` to hide on return.

6.2 Position-label bug — visible credibility hit

On the homepage Top Performers list, on the player search results, and on the matchup lineup disclosures, Vinícius Júnior is labelled "Full Back". On his player detail page, he is correctly labelled "Left Winger" — but the page header reads "Brazil · Left Winger · Left Winger7.99/10" (duplicated position with no space before the rating). Messi is labelled "Centre Forward" on the homepage and "Right Winger" on his detail page. The "Rank among Centre Forwards" / "Rank among Full Backs" line on the detail page reveals the truth: the list-page label is the **position-group used for percentile ranking**, not the player's actual position. This is misleading — Vinícius is not a Full Back by any reasonable definition. Any soccer fan will spot this immediately and the credibility hit is severe. **Fix:** either show the real position in the list (pull `position_detail` from `identity_players`, not the percentile-rank bucket), or rename the rank-group field ("Rank among forwards" not "Full Back"). Also fix the duplicated-position rendering in the detail-page header.

6.3 Jargon inventory — the worst offenders

The full jargon inventory runs to 20+ terms. The table below lists the worst offenders — terms that appear on the live site without inline definition, with their first-occurrence file:line and a suggested plain-English gloss. The Tooltip component exists; it just needs wiring.

Term (as it appears on site)	First appears at	Suggested plain-English gloss
xG	RawStats.tsx:148	Expected Goals — chance a shot scores, 0 to 1. 0.5 xG ≈ coin-flip finish.
xA	RawStats.tsx:154	Expected Assists — chance a pass becomes a goal, 0 to 1.
Per 90 (toggle)	RawStats.tsx:116	Stats scaled to a full 90-minute match, so subs compare fairly to starters.
Sofascore rating	MatchTimeline.tsx:118	Match rating 0-10 from Sofascore (a football stats site). 7+ good, 6 OK, <6 poor.
Monte Carlo (10,000 sims)	bracket/page.tsx:20	We play out the rest of the WC 10,000 times, counting how often each nation wins.
Chaos / avg match entropy	BracketGrid.tsx:140	Upset risk: 0% = every match a lock, 100% = every match a coin flip.
Brier Score	brier/SummaryCards.tsx:96	How far off our predictions were. Lower = better. Coin-flip = 0.25.
Δ Brier	MatchLogTable.tsx:235	Model edge — how much better our prediction was than the bookies'. Positive = we won.
Calibration	brier/page.tsx:16	How well our predicted %s match real outcomes. A "70%" call should be right 70% of the time.
FIFA-style rating (89 Elite)	FifaBadge.tsx:53	0-99 score in FIFA video-game style. 90+ World Class, 85-89 Elite, 80-84 Top Tier.
OpenRouter (vendor leak)	TacticalAnalysis.tsx:79	Drop "OpenRouter"; say "AI-generated" + tooltip "Written by an LLM fed the player's stats."

Term (as it appears on site)	First appears at	Suggested plain-English gloss
Understat (data source)	RawStats.tsx:224	Move to a "Data sources" footer chip; don't cite inline next to stat tiles.
Percentile	PlayerRadar.tsx:69	Where this player ranks among peers at their position. 90th = better than 90% of peers.
Title prob / Title Favorites	HomeCards.tsx:85	Chance of winning the World Cup. Already the HomeCards subtitle — bracket "%" needs same tooltip.
Value Pick / Value Edge	HomeCards.tsx:251	Where our model disagrees with the bookies by 3+ points — we think the team is underrated.

6.4 World Cup 2026 context fit — lean into the knockout narrative

The 2026 World Cup is the first tri-hosted World Cup (USA / Mexico / Canada) across 16 cities. The backend has `data/static/venues_2026.json` (referenced in `BOARD.md`, and the attached PR plan's PR 5.3 adds rest/travel modeling from it) but the frontend surface of venue / host-city flavor is minimal. The only WC 2026 flavor in the frontend today is the string "WC 2026 Intelligence" in the sidebar footer. Given the deliberate knockout-only scope (§5.2), the opportunity is not to add group-stage infrastructure but to **lean harder into the knockout narrative**: venue names on MatchCard (the data exists), a tri-host flag lockup in the sidebar header, and an explainer ribbon above the bracket ("32 teams: 24 group qualifiers + 8 best third-place teams — here's how the bracket shapes up").

The Title Favorites card on the homepage needs context regardless of scope. Top team is 6.4% — statistically under-confident vs bookmaker consensus (~12-18% for the favorite). Argentina is absent from the top 5 despite being defending champions. The Value Pick card reads "Cape Verde vs Argentina, +34% edge, model 45% · bookies 12%" — implying TrueScout gives Cape Verde a 45% chance to beat Argentina, an extraordinary possibly model-broken claim presented without any hedging or context. If Argentina wins 5-0 (likely), trust collapses. **Fix**: add a "storyline" tag per team in the top-5 (reigning champion, host nation, rising overnight delta > 1pp); hedge the Value Pick card ("high-variance call — model disagreement with market"). These are content changes, not infrastructure — ship during the tournament.

SECTION 7 · VS SOFAScore / FOTMOB / FBREF / UNDERSTAT

Part V — Competitive Benchmark

TrueScout is not trying to be SofaScore and should not try. The differentiation story is real: it is the only World Cup product that combines transparent Bayesian player ratings, 10,000-run Monte Carlo tournament simulations, a model-vs-bookmaker value-pick engine, and a public calibration dashboard — and shows its math on a dedicated About page. The trade-off is real too: it is WC-only, has no live match tracker, no heatmaps, no group stage, and currently no live bookmaker data. It is a complementary "second screen for prediction nerds", not a SofaScore replacement.

Capability	TrueScout	SofaScore	FBref	Understat
Bayesian player ratings with explicit confidence tiers (High/Moderate/Sparse)	✓	—	—	—
10,000-run Monte Carlo tournament sims with alternative-team expansion per matchup	✓	partial (bracket predictor only)	—	—
"Chaos meter" (avg match entropy) per round — unique, fun	✓	—	—	—
Value picks flagging model-vs-bookie divergence	✓	—	—	—
AI-generated scouting reports per player (currently 500ing)	✓	—	—	—
Side-by-side player Compare with plain-English verdict	✓	—	compare tool, no NL verdict	—
Public calibration dashboard (Brier) tracking prediction accuracy vs bookies	✓	—	—	—
Honest limitations page (About section 4)	✓	—	—	—
Live match tracker with real-time events	—	✓	✓	—
Player heatmaps, touch maps, shot maps	—	—	✓	✓
Group stage standings and tables	—	✓	—	—
Head-to-head team history	—	✓	—	—
Top scorers / golden boot race tables	—	✓	—	partial
Sortable, filterable player tables	—	—	✓	—
Live odds comparison across multiple bookmakers	—	✓	—	—
Native mobile apps + responsive web	—	✓	✓	✓
News feed, video highlights, social integration	—	✓	✓	—
League / competition breadth (not WC-only)	—	✓	✓	✓

FotMob is omitted from the table to keep column count readable — it overlaps SofaScore on live tracker, news, and matchday features, and lacks TrueScout's Bayesian / Monte Carlo / calibration layer.

Differentiation story for fans

Casual fans who want "who's going to win?" get a defensible answer with a value-pick call-out. Nerds who want to argue methodology get the limitations page and the Brier tracker. The mobile experience is too broken today to be any fan's primary device. The winning move is to lean into the differentiation — publish the Brier delta, surface the AI scouting reports, add Compare to the nav — and explicitly position as **"the prediction layer"** rather than "the everything app". The worst outcome would be to chase SofaScore's feature breadth and lose the prediction-layer differentiation that makes TrueScout worth opening in the first place.

SECTION 8 · WHAT IS GENUINELY WELL-BUILT

Top 3 Wins

Rigorous critique does not mean ungenerous. Three things in this codebase are genuinely senior-grade and would be wins at any company.

W1. Backend: data-driven bracket + vectorised Monte Carlo + anti-hallucination LLM

The `_load_bracket` function (*monte_carlo_sim.py:109-297*) reads the 16 actual R32 fixtures and 8 R16 fixtures from Bronze ESPN Parquets, parses the "Round of 32 N Winner" placeholder strings to derive the bracket pairing tree, repairs orphaned R32 indices when ESPN emits stale placeholders, and asserts the bracket has exactly 32 unique teams. The bracket auto-updates as matches are played and ESPN commits new fixtures, with zero hardcoding. The 10,000-iteration Monte Carlo (*monte_carlo_sim.py:521-535*) is vectorised in pure NumPy using `np.broadcast_to` and fancy indexing — no Python loop over iterations, only over the 5 rounds — and runs in ~0.04 s. The `_validate` function explicitly checks `title_prob sum = 1.0 ± 0.02` and raises `RuntimeError` if not — a hard correctness gate before DB write. The LLM scouting reports (*generate_narratives.py:64-175*) construct a structured "Facts" payload from the player's Bayesian metrics, and the system prompt explicitly bans the LLM from inventing any number not in the bullets; the `_JARGON_BAN` constraint bans "posterior", "HDI", "Bayesian", "shrinkage" — forcing the LLM to write as a TV analyst, not a stats PhD. Confidence-gated routing sends low-confidence players to a "Traditional Scout" voice forbidden from mentioning stats at all. **This is the right way to use an LLM in a factual product.**

W2. Frontend: loading skeletons on every route + bracket geometry

Every single route segment has a dedicated, layout-matched skeleton (*app/loading.tsx*, *app/players/loading.tsx*, *app/players/[reep_id]/loading.tsx*, *app/matchups/loading.tsx*, *app/bracket/loading.tsx*, *app/brier/loading.tsx*). This is rare and senior-grade — most Next.js apps skip this. The skeletons preview the actual content shape (card grids, two-column profile, table rows), not generic spinners. The bracket geometry (*BracketGrid.tsx:9-67*, *lib/bracket.ts:65-233*) uses a flex-based layout where slot flex doubles each round and connector arms anchor at 25%/75% of each group — an elegant solution that avoids absolute-positioning hell. `buildBracket` correctly walks R32→R16→QF→SF→F, resolves slot winners from the joint distribution with marginal-prob fallback, computes a chaos (binary-entropy) score per round, and surfaces alt dark-horse teams per slot. The data flow is clean and the alt-team disclosure is a genuinely nice touch for upset-curious fans.

W3. Live site: About page is a masterclass + Compare delivers plain-English verdicts

The About page (*/about*) explains Bayesian shrinkage with a "weighted average" analogy, defines the confidence tiers (High/Moderate/Sparse → green/amber/red), frames Brier Score ("lower = better, coin-flip = 0.25"), and lists **five honest limitations**: small WC sample, Sofascore is a black box, no injury feed, penalties are 50/50, no tactical/manager signal. Most analytics products hide their methodology; TrueScout publishes it. This is the single biggest trust differentiator on the live site. The Compare tool (*/compare*) delivers

plain-English verdicts like "Lionel Messi has a marginal edge (0.24 pts) — both players are broadly comparable." Turns eight side-by-side metrics into a take-away sentence. FBref's compare tool does not do this. The verdict quality is genuinely senior-grade. Loading-state polish across every route makes the site *feel* premium even where features are thin.

SECTION 9 · CROSS-REFERENCED AGAINST THE IN-FLIGHT PR PLAN

Top 5 Fixes — After Overlap Check

The attached PR plan (PR1-PR10, four rounds of development) already covers most of this audit's recommendations. After cross-referencing, only **three items** remain as genuinely new work for the solo developer this week. The other two original fixes (AI 500 hide, position-label + jargon tooltips) are already in-flight and checked off below. This section lists what's already covered, then the remaining gaps, then the specific "how to do each" for the three that are not yet addressed.

9.1 Already in-flight (from the PR plan — checked off)

The following items from this audit's original recommendation set are already addressed by the attached PR plan. No additional action needed — just verify they ship.

Audit recommendation	PR plan ref	How the plan addresses it	Status
Fix/hide AI Scouting Report 500	PR 5d (urgent)	Vercel runtime config (maxDuration=60), reasoning-tag stripping, 4-model fallback chain, players.json module-level cache, pre-generation as primary path. Thoroughly addressed.	✓ In-flight
Position-label bug fix (Vini Jr as "Full Back")	PR 1.1	Extends _REEP_BUCKET, modal Sofascore position aggregation, UNK fallback bucket, position_source audit column, GK/xG sanity asserts. Root-cause fix.	✓ In-flight
Plain-English label pass (xG, xA, Posterior Mean → Overall Rating, etc.)	PR 2.1 + PR 5c.1	Comprehensive rename table across all page.tsx files. PR 5c.1 specifically strips jargon from the Rating Breakdown card that PR 2.1 missed.	✓ In-flight
Shared Tooltip component + sitewide wiring	PR 2.2	Tooltip.tsx created and applied to every renamed label. (Audit note: verify LabelWithInfo icon becomes a keyboard-focusable button — see §5.4 a11y finding.)	✓ In-flight
Frontend team_aliases mirror	PR 5c.3	New frontend/lib/teamAliases.ts mirrors the canonical ETL module. ETL exports team_aliases.json at build time so they cannot drift.	✓ In-flight
Bookmaker odds ingestion (Brier page value prop)	PR 5c.2	market_odds_archive table + ESPN pull ingestion + per-day parquet snapshots. Fallback chain: live odds → archive → brier_log.	✓ In-flight
Compare feature (the hidden flagship)	PR 8.1	Full /compare route with overlay radar + delta-bar table + AI narrative. Linked from player profiles, search, matchup cards.	⚠ Partial — verify sidebar nav entry

Audit recommendation	PR plan ref	How the plan addresses it	Status
Hyperparameter calibration / sensitivity CLI	PR 5.4	Grid search PRIOR_PULL × OPPONENT_ALPHA × LOGISTIC_SCALE, weekly run, persists to calibration_params.json. Addresses the "uncalibrated magic numbers" finding (§3.5).	✓ In-flight
Golden Boot / Golden Ball projections	PR 9.3	Monte Carlo per-player xG accumulation during 10k iterations. Top 20 candidates with median + 90% CI. New awards.json.	✓ In-flight
Methodology / limitations page (About)	PR 2.6 + PR 5.6	About page shipped with 5 honest limitations. PR 5.6 expands with model-fit caveats. Audit verified this is genuinely excellent live.	✓ Shipped
Raw stats panel + per-match timeline	PR 2.3 + PR 2.4	RawStats.tsx + MatchTimeline.tsx with per-90 toggle, position-group comparators, color-coded Sofascore ratings. Audit verified these exist on the live site.	✓ Shipped
Rest days + travel adjustment	PR 5.3 + PR 5c.5	venues_2026.json + haversine travel_km + rest-day penalty in Monte Carlo. Rest days display landed; travel km on MatchCard is PR 5c.5.	⚠ Partial — travel km pending

9.2 Remaining gaps — the three items NOT in the PR plan

After cross-referencing, three findings from this audit are **not** addressed by the PR plan. These are the only items that need the developer's time this week — total ~12 hours of focused work, not the ~24-32 hours from the original audit.

#	Gap not covered by PR plan	Impact	Effort	Severity	Files
1	Mobile sidebar + hamburger (the plan explicitly defers this as "responsive enough" — but live-site measurements show 224px sidebar on 375px viewport, 973px bracket overflow)	Huge	M (~6h)	Critical	Sidebar.tsx:64, layout.tsx:31
2	Add Compare to sidebar nav (PR 8.1 builds /compare but does not add a sidebar entry — the feature stays invisible without this)	High	S (~1h)	Critical	Sidebar.tsx
3	Fix Brier 90-min-draw skip + backend alias de-dup (PR 5c.2 fixes bookmaker data, PR 5c.3 fixes frontend aliases — but the Brier draw-skip and the stale backend alias dicts in brier_tracker.py:65-75 / matchups.py:59-67 are not covered)	High	S (~4h)	Critical (math)	brier_tracker.py:250-257, brier_tracker.py:65-75, matchups.py:59-67

9.3 How to do each remaining fix

Gap 1 (mobile sidebar — the plan says "responsive enough", the data says otherwise): the attached PR plan explicitly lists "Mobile-first redesign" as a non-goal, stating "current Tailwind layout is responsive enough." The live-site audit measured otherwise: on a 375px iPhone 12, the sidebar renders at a fixed 224px

width, leaving only 55px of usable content width. The bracket page overflows by 973px. This is not a redesign — it is a collapse. Create `app/components/SidebarClient.tsx` with `useState(isOpen)`. Render `<aside className="hidden lg:flex w-56">` for desktop and a hamburger button + drawer overlay for mobile. Use `usePathname()` to auto-close on navigation. Budget 6 hours including mobile QA. This does not contradict the plan's "no mobile-first redesign" non-goal — it is a targeted fix for a broken layout, not a redesign.

Gap 2 (Compare in sidebar nav — PR 8.1 builds the feature but not the nav entry): the PR plan's PR 8.1 creates the `/compare` route with overlay radar, delta-bar table, and AI narrative. It specifies linking from player profiles, search, and matchup cards — but does not mention adding a sidebar nav entry. Without a sidebar entry, the feature is still invisible to any user who does not click a "Compare" button somewhere else first. Add a sidebar entry between "Player Search" and "Prediction Tracker": `{ icon: scale, label: "Compare", sublabel: "Side-by-side player verdicts", href: "/compare" }`. Estimated 1 hour. This is the smallest highest-ROI change in the audit.

Gap 3 (Brier draw-skip + backend alias de-dup — not in the PR plan): two backend issues the PR plan does not address. (a) In `brier_tracker.py:250-257`, 90-minute draws in knockout matches are skipped entirely (`continue`) instead of being resolved via ET/pens scores that already exist in the Sofascore `EVENTS_SCHEMA` (`sofascore_pull.py:184-189` has `home_score_et`, `away_score_et`, `home_score_penalties`, `away_score_penalties`). This biases the Brier metric toward blowouts. (b) `brier_tracker.py:65-75` and `matchups.py:59-67` each define their own stale 7-entry `_NAME_ALIASES` dict — a subset of the canonical 14-entry `TEAM_ALIASES` in `etl.utils.team_aliases`. Missing entries include "Czechia" and "Holland". PR 5c.3 mirrors the frontend but does not fix the backend stale copies. Fix: replace the `continue` with ET/pens resolution logic; delete the two stale dicts and import from the canonical module. Pick one ET-bias convention (0.55/0.45 is the better one) and apply it everywhere. Estimated 4 hours.

SECTION 10 · SHIP DURING THE WORLD CUP, NOT AFTER

Tournament-Window Roadmap

The tournament IS the timeline. The original audit deferred too aggressively to "post-tournament V2". Revised approach: three buckets. (1) Already in-flight via the PR plan — verify they ship. (2) Remaining gaps from §9.2 — ~11 hours this week. (3) Tournament-safe items pulled back from the deferred list — additive changes that do not touch the math core and can ship safely during the World Cup. The math core changes (WC-floor removal, uncertainty propagation) stay deferred because changing title probabilities mid-tournament without a calibration run is riskier than the status quo.

10.1 This week — the three remaining gaps (~11 hours)

Day	Task	Files	Effort	Severity
Day 1-2	Mobile sidebar + hamburger (Gap 1)	Sidebar.tsx, layout.tsx, new SidebarClient.tsx	~6h	Critical

Day	Task	Files	Effort	Severity
Day 3	Add Compare to sidebar nav (Gap 2)	Sidebar.tsx	~1h	Critical
Day 4-5	Brier draw-skip fix + backend alias de-dup (Gap 3)	brier_tracker.py:250-257, brier_tracker.py:65-75, matchups.py:59-67	~4h	Critical (math)

Total: ~11 hours over 5 days. The rest of the week is buffer for mobile QA, verifying the PR-plan items ship, and picking up tournament-safe items from §10.3.

10.2 Tournament-safe — pull back from deferred (ship during WC, ~12h)

These items were originally deferred to "post-tournament V2" but can safely ship during the World Cup. They are additive (do not change existing math outputs), low-risk (no visual regression on core pages), and each is independently shippable. Pick them up after the three gaps in §10.1 are closed, or in parallel if time allows.

Tournament-safe item	Files	Effort	Severity	Touch math core?
Replace th onClick with keyboard-accessible sort buttons + aria-sort	MatchLogTable.tsx, Tooltip.tsx	~4h	Major (a11y)	No — pure a11y improvement on existing table
Parameterised SQL in load_identity._apply_position_overrides	load_identity.py:159-165	~1h	Major (security)	No — not exploitable today, just good hygiene
Alerting on nightly failure (Slack/Discord webhook)	run_nightly.py	~2h	Major (ops)	No — additive; does not touch the pipeline logic
Remove dead deps (pymc, apscheduler, rich) + pin critical-path deps	requirements.txt	~1h	Minor (hygiene)	No — removes unused imports; verify CI passes
Move fbref_pull.py to _archive/; delete empty etl/matching/ stub	etl/sources/, etl/matching/	~30m	Minor (hygiene)	No — dead code cleanup
Hedge Value Pick card + add storyline tags to Title Favorites	HomeCards.tsx	~1h	Major (credibility)	No — content change, not math change
Add venue names to MatchCard from venues_2026.json	MatchCard.tsx	~1h	Major (WC26 context)	No — additive display field
PWA wrapper (manifest.json + service worker for offline-tolerant nav)	next.config.ts, manifest.json	~4h	Differentiating	No — additive; does not change any route logic

10.3 Stay deferred — math core changes (ship after the tournament)

These items genuinely need to wait. They change the title probabilities or the rating outputs mid-tournament, which requires a calibration run against ≥ 20 graded matches and a regression check against the existing Brier log. Shipping them without validation is riskier than the status quo — a mid-tournament change to "France

7.1% to win" that shifts to "France 9.8%" overnight erodes trust more than the current WC-floor hack does. The PR plan's PR 5.4 (hyperparameter calibration, weekly grid search) will produce the calibration data needed to validate these changes. Ship them in the first week after the tournament final.

Deferred item	Files	Severity	Why it must wait
Remove WC-floor override (M1)	monte_carlo_sim.py:321-329	Critical (math)	Changes all title probabilities; needs calibration run vs PR 5.4 output
Propagate per-player posterior uncertainty into Monte Carlo (M5)	monte_carlo_sim.py:521-535	Critical (math)	Changes title prob widths; needs regression check on Brier log
Refactor monolithic export_json.py (1,388 lines) into etl/export/*	etl/export/ (new package)	Major (DX)	High-risk refactor during tournament — one bad merge breaks the nightly export
Replace recharts radar with hand-rolled SVG polygon	PlayerRadar.tsx	Major (perf)	Visual regression risk on the most-visited player profile page
Live match tracker MVP, H2H team pages, full i18n	multiple new routes	Differentiating	V2 scope — each is a 1-2 week project on its own

10.4 What NOT to build (ever)

Resist the urge to chase SofaScore's feature breadth. Do not build: native iOS / Android apps (PWA is enough), video highlights (rights nightmare), news feed (commoditised), league/competition breadth beyond WC26 (dilutes focus), transfer values / contract info (different product). Each is a 6-month project that would pull the team away from the prediction-layer differentiation that makes TrueScout worth opening.

SECTION 11 · RUBRIC, JARGON INVENTORY, CODE SNIPPETS

Appendix

A. Weighted Composite Score

The composite is computed as a weighted mean across the three audit layers, with fan-fit weighted 2× because it is the product's stated primary lens. Backend scores are weighted 30%, frontend 30%, live-site 30%, fan-fit (cross-cutting) 10% × 2.

Layer	Avg score	Weight	Weighted
Backend (math + engine + pipeline + API + audience-fit)	5.8	30%	1.74
Frontend (design + onboarding + mobile + perf + a11y + WC26 fit + craft)	4.1	30%	1.23

Layer	Avg score	Weight	Weighted
Live site (first impression + mobile + onboarding + polish + IA + trust)	4.7	30%	1.41
Fan-fit (cross-cutting: onboarding + mobile + jargon + WC26 context)	3.2	10% × 2	0.64
Composite (sum)			5.02 ≈ 4.9

B. Code Snippets Compendium

Six verbatim excerpts from the codebase that illustrate either a great pattern or a critical issue. Each is cited to file:line and accompanied by one-line commentary.

Snippet 1 — The Bayesian update (correct)

```
# etl/models/bayesian_ratings.py:310-323
tau_prior = 1.0 / sigma2_prior
tau_wc     = np.where(np.isinf(sigma2_wc), 0.0, 1.0 / sigma2_wc)
tau_post   = tau_prior + tau_wc

mu_prior = df["prior_mean"].values
wc_obs_col = "wc_rating_adjusted" if "wc_rating_adjusted" in df.columns else "wc_rating_avg"
mu_wc = df[wc_obs_col].fillna(df["wc_rating_avg"]).fillna(df["prior_mean"]).values

# Posterior
mu_post = (tau_prior * mu_prior + tau_wc * mu_wc) / tau_post
sigma2_post = 1.0 / tau_post
```

Normal-Normal conjugate: precisions add, posterior mean is precision-weighted avg. Textbook partial pooling. Correct.

Snippet 2 — The WC-floor hack (broken)

```
# etl/models/monte_carlo_sim.py:321-329
wc_floor_expr = f"""
GREATEST(
    pr.posterior_mean,
    CASE WHEN pr.wc_minutes >= 90 AND f.wc_rating_avg IS NOT NULL
        THEN f.wc_rating_avg * 0.90
        ELSE pr.posterior_mean
    END
)"""
```

The single most consequential line in the codebase. Silently overrides the Bayesian posterior. Every title probability flows through this.

Snippet 3 — The Brier draw-skip (biased)

```
# etl/models/brier_tracker.py:250-257
else:
    # 90-min draw – in knockout this means ET/pens; scores reflect 90 min
    # We don't have the ET/pens result in Bronze yet – log and skip
    logger.warning(
        "%s vs %s ended %d-%d (90 min draw) – ET/pens result not in Bronze yet, skip
ping.",
        home, away, h_score, a_score,
    )
    continue
```

Comment says ET/pens data is missing — but sofascore_pull.py:184-189 already captures it. The continue biases the Brier metric.

Snippet 4 — Vectorised Monte Carlo core (elegant)

```
# etl/models/monte_carlo_sim.py:521-535
for round_idx in range(1, n_rounds):
    n_alive = current.shape[1]
    n_matches = n_alive // 2

    left = current[:, 0::2]    # (n_sim, n_matches)
    right = current[:, 1::2]

    s_left = strengths[left]   # NumPy fancy indexing
    s_right = strengths[right]

    p_left = 1.0 / (1.0 + np.power(10.0, -(s_left - s_right) / scale))
    rand = rng.random((n_sim, n_matches))

    winners = np.where(rand < p_left, left, right)
    current = winners
```

10,000 bracket iterations in one NumPy expression per round. The only Python loop is over the 5 rounds. Caveat: `strengths` is a deterministic 32-vector, not a posterior sample (see §3.4).

Snippet 5 — Title-prob sum invariant (defensive numerics, correct)

```
# etl/models/monte_carlo_sim.py:677-680
total = title_rows["title_prob"].sum()
logger.info("Title prob sum = %.6f (expect 1.0)", total)
if abs(total - 1.0) > 0.02:
    raise RuntimeError(f"Title probs sum to {total:.4f} – check simulation logic.")
```

Hard correctness gate before DB write. Catches off-by-one bracket errors. The verify_outputs.py step repeats this check on the exported JSON. Senior-grade defensive numerics.

Snippet 6 — The broken mobile layout (three lines)

```
// app/layout.tsx:31-33 – the broken mobile layout in three lines

{children}
```

renders a fixed `w-56` aside with no `hidden lg:flex`, no hamburger. On 375px viewport the sidebar (224px) leaves only 55px of actual content width. Most consequential three lines in the codebase.

C. Closing Note

This audit was conducted against the public repo and live site as of 2026-07-01, then revised after cross-referencing the attached in-flight PR plan (PR1-PR10). The good news: most of the original recommendations are already being addressed — position-label fix, jargon tooltips, AI 500 stabilization, bookmaker odds ingestion, Compare feature, hyperparameter calibration, and Golden Boot projections are all in-flight. The three remaining gaps (mobile sidebar, Compare-in-nav, Brier draw-skip + backend alias de-dup) total ~11 hours and are the only genuinely new work this week. A further ~12 hours of tournament-safe items (allly sort buttons, nightly alerting, dead dep cleanup, Value Pick hedging, venue names, PWA wrapper) can ship during the World Cup without touching the math core. The math core changes (WC-floor removal, uncertainty propagation) stay deferred until the PR plan's PR 5.4 hyperparameter calibration produces enough data to validate them — shipping title-probability changes mid-tournament without calibration is riskier than the status quo. The composite 4.9 / 10 is not a verdict on the team — it is a snapshot of the gap between ambition and current execution. Close the three gaps in §9.2 and the score lifts to ~6.5. Ship the tournament-safe items in §10.2 and it lifts to ~7. The ceiling here is genuinely high; a single focused week closes most of the gap.